

THE SYSTEM SHALL send an email confirmation to the user after they successfully register an account.

THE SYSTEM SHALL send a notification to the user if a helper has accepted a request

THE SYSTEM SHALL send a notification to available helpers if a user requests

THE SYSTEM SHALL make sure only necessary information gets sent

THE SYSTEM SHALL give the user the ability to cancel specific helpers

THE SYSTEM SHALL cancel the extra helpers if too many accept

As a developer I want the system to be developed in on big python file

As a user I want only my needed info to be shared to the helpers

As a user I want the system to give me an approximate wait time if no helpers are available

the system shall let the user add current location, where they want to go and what time

What you've written is a mix of system requirements and user stories, but it's a bit inconsistent and overlapping. Let's clean it up into something clearer and more structured so it's actually usable for development.

✅ Refined System Requirements

Notifications & Communication

- The system shall send an email confirmation after a user successfully registers an account.
- The system shall notify a user when a helper accepts their request.
- The system shall notify available helpers when a new user request is created.

Data Privacy

- The system shall ensure that only necessary user information is shared with helpers.
- The system shall allow users to control what information is shared with helpers.

Request & Helper Management

- The system shall allow users to cancel specific helpers assigned to their request.
- The system shall automatically cancel excess helpers if more helpers accept than needed.

Availability & Feedback

- The system shall provide users with an approximate wait time if no helpers are currently available.

User Input

- The system shall allow users to input:
 - Current location
 - Destination
 - Desired time



Refined User Stories

- As a developer, I want the system implemented in a single Python file so that it is simple to deploy and manage. *(Note: this is usually not scalable, but acceptable for a prototype.)*
- As a user, I want only necessary information shared with helpers so that my privacy is protected.
- As a user, I want to receive an estimated wait time if no helpers are available so that I can plan accordingly.



A quick reality check

Keeping everything in **one big Python file** will become messy fast if this grows beyond a prototype. Even a small system like this benefits from separating:

- user logic
- notification system
- helper matching
- data handling

If this is just for a school project or MVP, fine. Otherwise, you'll regret it pretty quickly.

If you want, I can turn this into:

- ↳ UML diagrams
- ↳ a database schema
- or ↳ actual Python starter code

Just tell me 👍



Give me full python code for that system in one singular file

Helper Matching System (single File Python) · pytl

Kopiera Redigera

Ladda
ner

Kör
koden

```
import time
import threading
from datetime import datetime, timedelta

# -----
# Simple in-memory data storage
# -----
users = {}
helpers = {}
requests = {}

# -----
# Utility functions
# -----
def send_email(user_email, message):
    print(f"[EMAIL to {user_email}]: {message}")

def send_notification(user_id, message):
    print(f"[NOTIFICATION to {user_id}]: {message}")

# -----
# Models (simple dict-based)
# -----
def register_user(user_id, email):
    users[user_id] = {
        "email": email,
        "shared_info": {"location": True, "destination": True, "time": True
```

```
}
send_email(email, "Registration successful!")

def register_helper(helper_id):
    helpers[helper_id] = {
        "available": True
    }

# -----
# Request Handling
# -----
def create_request(user_id, location, destination, time_needed):
    request_id = len(requests) + 1
    requests[request_id] = {
        "user_id": user_id,
        "location": location,
        "destination": destination,
        "time": time_needed,
        "helpers": [],
        "status": "waiting"
    }

    notify_helpers(request_id)
    estimate_wait_time(request_id)

    return request_id

def notify_helpers(request_id):
    for helper_id, helper in helpers.items():
        if helper["available"]:
            send_notification(helper_id, f"New request available: {request_

def helper_accept_request(helper_id, request_id):
    req = requests.get(request_id)

    if not req:
        return

    if len(req["helpers"]) >= 1:
        send_notification(helper_id, "Request already has enough helpers. C
        return

    req["helpers"].append(helper_id)
    helpers[helper_id]["available"] = False

    send_notification(req["user_id"], f"Helper {helper_id} accepted your re

    # Cancel extra helpers automatically
    cancel_extra_helpers(request_id)
```

```
def cancel_extra_helpers(request_id):
    req = requests[request_id]

    if len(req["helpers"]) > 1:
        extra_helpers = req["helpers"][1:]
        req["helpers"] = req["helpers"][:1]

        for helper_id in extra_helpers:
            helpers[helper_id]["available"] = True
            send_notification(helper_id, "You were removed due to excess he

def cancel_specific_helper(user_id, request_id, helper_id):
    req = requests.get(request_id)

    if not req or req["user_id"] != user_id:
        return

    if helper_id in req["helpers"]:
        req["helpers"].remove(helper_id)
        helpers[helper_id]["available"] = True
        send_notification(helper_id, "You were cancelled by the user.")

# -----
# Privacy Control
# -----
def get_shared_info(user_id, request_id):
    user = users[user_id]
    req = requests[request_id]

    info = {}
    if user["shared_info"]["location"]:
        info["location"] = req["location"]
    if user["shared_info"]["destination"]:
        info["destination"] = req["destination"]
    if user["shared_info"]["time"]:
        info["time"] = req["time"]

    return info

# -----
# Wait Time Estimation
# -----
def estimate_wait_time(request_id):
    available_helpers = sum(1 for h in helpers.values() if h["available"])

    if available_helpers == 0:
        wait_time = "Approx. 10-15 minutes"
    else:
```

```

        wait_time = "Less than 5 minutes"

    user_id = requests[request_id]["user_id"]
    send_notification(user_id, f"Estimated wait time: {wait_time}")

# -----
# Example Simulation
# -----
if __name__ == "__main__":
    # Register users and helpers
    register_user("user1", "user1@example.com")
    register_helper("helper1")
    register_helper("helper2")

    # Create request
    req_id = create_request("user1", "Point A", "Point B", "12:00")

    # Helpers accept request
    helper_accept_request("helper1", req_id)
    helper_accept_request("helper2", req_id)

    # User cancels helper
    cancel_specific_helper("user1", req_id, "helper1")

    # Check shared info
    print("Shared info:", get_shared_info("user1", req_id))

```

I've put a complete single-file Python prototype in the canvas.

A few important notes before you run it:

- It's an **in-memory simulation** (no database, no real email/notifications).
- "Email" and "notifications" are just printed to the console.
- It enforces:
 - registration confirmation
 - helper notifications
 - helper acceptance + auto-cancel of extras
 - user-controlled helper cancellation
 - basic privacy filtering
 - simple wait-time estimation

If you want, I can extend this into:

- ↳ a Flask/FastAPI backend
- ↳ real email sending (SMTP)

- ↳ a matching algorithm based on distance
- or ↳ splitting it into proper modules (when you're ready to move beyond the "one file" constraint)

